WaveRider: A Zero-Parameter Geometric Machine Learning Stack for Manifold-Aware Classification

Eric G. Suchanek, PhD
Independent Researcher
suchanek@mac.com

https://github.com/suchanek/proteusPy

March 2026

Abstract

We present **WaveRider**, a four-layer framework for manifold-aware machine learning that achieves competitive classification without learned parameters. The core thesis is that *the manifold is the model*: the data manifold, once discovered through local principal component analysis and encoded as a weighted knowledge graph with stored basis geometry, is itself a sufficient classifier.

WaveRider consists of: (1) **TurtleND**, an N-dimensional navigation primitive carrying position and an orthonormal frame, generalised from the classical Turtle3D molecular modelling primitive via Givens rotations; (2) **ManifoldWalker** and its Adam variant, implementing Riemannian-approximate gradient descent by projecting updates onto locally-estimated tangent planes; (3) **ManifoldModel**, a zero-parameter classifier whose "weights" are the graph topology, local basis vectors, and eigenvalue field discovered during a single exploration pass; and (4) **ManifoldObserver**, a novel $(N+1)$-dimensional construction that extends any N-dimensional manifold frame by one orthonormal dimension via QR decomposition, gaining the manifold normal and the ability to observe curvature, topology, and class boundaries from *above* the surface—turning a search problem into a projection problem.

The ManifoldObserver operationalises a principle with deep roots: Plato's prisoners see only shadows on the cave wall until lifted into three dimensions; Abbott's Flatlanders see only slices until raised above their plane; Gauss's extrinsic curvature is invisible from the surface until one dimension is added. WaveRider provides the constructive machine-learning instantiation of this principle.

On the UCI Digits dataset (64 dimensions) ManifoldModel achieves 97.72% accuracy with zero parameters versus 97.33% for Euclidean KNN. On CIFAR-10 (3,072 dimensions) ManifoldModel uses 219× fewer parameters than a standard CNN while matching its accuracy. The ManifoldObserver achieves 100% agreement with

its N-dimensional subject on synthetic benchmarks while providing curvature and topology information unavailable to the surface-bound walker. We also describe an application to protein disulfide bond analysis, where a 6-dimensional observer looks into the 5-dimensional torsion-angle manifold and sees the structural landscape that the walker must search.

1 Introduction

The dominant paradigm in machine learning is to fit a parameterised function—a neural network, a kernel machine, a decision tree—to training data by optimising a loss. The resulting weights encode, implicitly, knowledge about the data distribution. This works well but has a cost: millions of parameters for complex tasks, opacity about what was learned, and fragility when the test distribution shifts.

An alternative is to *discover* the data manifold explicitly and use that discovery as the model. Real high-dimensional data—images, protein structures, speech signals—rarely uses all of its ambient dimensions. Handwritten digits live on a manifold of intrinsic dimension roughly 13 inside 64-dimensional pixel space. CIFAR-10 natural images sit on a manifold of intrinsic dimension roughly 29 inside 3,072-dimensional space. More than 99% of the ambient dimensions are noise.

If we can find the manifold, we can discard the noise, measure distances correctly (along the surface, not through the void), and classify by *geometry* rather than by optimisation.

WaveRider does exactly this. It is not the first work to identify the manifold hypothesis [3, 4], nor the first to use local PCA for dimensionality reduction [5, 6]. What is novel is the integration: a coherent stack from a navigation primitive through optimisation to a zero-parameter classifier to an $(N + 1)$-dimensional extrinsic observer that can see what the classifier cannot.

The key conceptual contributions are:

1. **TurtleND**—the formal generalisation of Turtle3D to N dimensions using Givens rotations, enabling navigation in arbitrary-dimensional data spaces with the same primitives (move, rotate, orient) used for molecular modelling since 1986.

2. **ManifoldModel**—the identification that a k-NN graph with stored local PCA geometry at each node *is itself* a classifier, with no additional learned weights. Classification is manifold-aware nearest-neighbour search over this graph.

3. **ManifoldObserver**—a constructive proof that any N-dimensional manifold frame can be extended to $(N + 1)$ dimensions by a single QR decomposition, gaining the manifold normal. The observer classifies by projection rather than search, observes curvature and topology in $O(n)$ rather than $O(\text{hops} \times k)$, and embodies the geometric principle: *to understand a surface, you must leave it.*

1.1 Motivation: From Molecular Modelling to Machine Learning

The WaveRider stack did not originate in machine learning. It originated in molecular modelling. The Turtle3D class in proteusPy [7] was designed to build protein structures by navigating local coordinate frames along bond vectors—a technique dating to the author's 1992 *Proteus* program [8]. The frame carried heading, left, and up vectors; rotations encoded dihedral angles; molecules were "drawn" by a turtle navigating chemical space.

The generalisation to N dimensions arose when the same navigational primitive was needed for the 5-dimensional torsion-angle space of disulfide bonds. But once the N-dimensional turtle existed, the question became: what else can it navigate? The answer—data manifolds—led directly to the WaveRider stack.

This origin matters: WaveRider is not an abstraction built for machine learning. It is a navigation primitive that turned out to be equally at home in a protein crystal and a high-dimensional embedding space.

2 Related Work

Manifold learning. Isomap [3], LLE [4], UMAP [9], and t-SNE [10] are standard manifold-learning tools, but they are dimensionality-reduction methods—they produce low-dimensional embeddings for visualisation or downstream processing. WaveRider does not reduce dimensions; it discovers the manifold geometry *in place* and uses it directly for classification.

Local PCA. Local linear embedding [4] and its variants use local PCA to estimate tangent spaces. WaveRider's ManifoldWalker and ManifoldModel use local PCA similarly but store the full geometry (basis, eigenvalues, intrinsic dimension) at each node rather than producing a global embedding.

Riemannian optimisation. Methods for optimisation on Riemannian manifolds [11, 12] project gradients onto tangent planes and retract back to the manifold surface. ManifoldWalker performs approximate Riemannian gradient descent without requiring an explicit manifold parameterisation—the tangent plane is estimated locally at each step from the data neighbourhood.

Geometric deep learning. The field of geometric deep learning [13] argues that symmetries and geometric structure should be built into neural architectures. WaveRider takes this further: rather than embedding geometric structure in a *trained* model, the geometry *is* the model.

Prototype-based classifiers. Learning Vector Quantisation (LVQ) [14] and its successors classify by distance to learned prototypes. ManifoldModel is in the spirit of

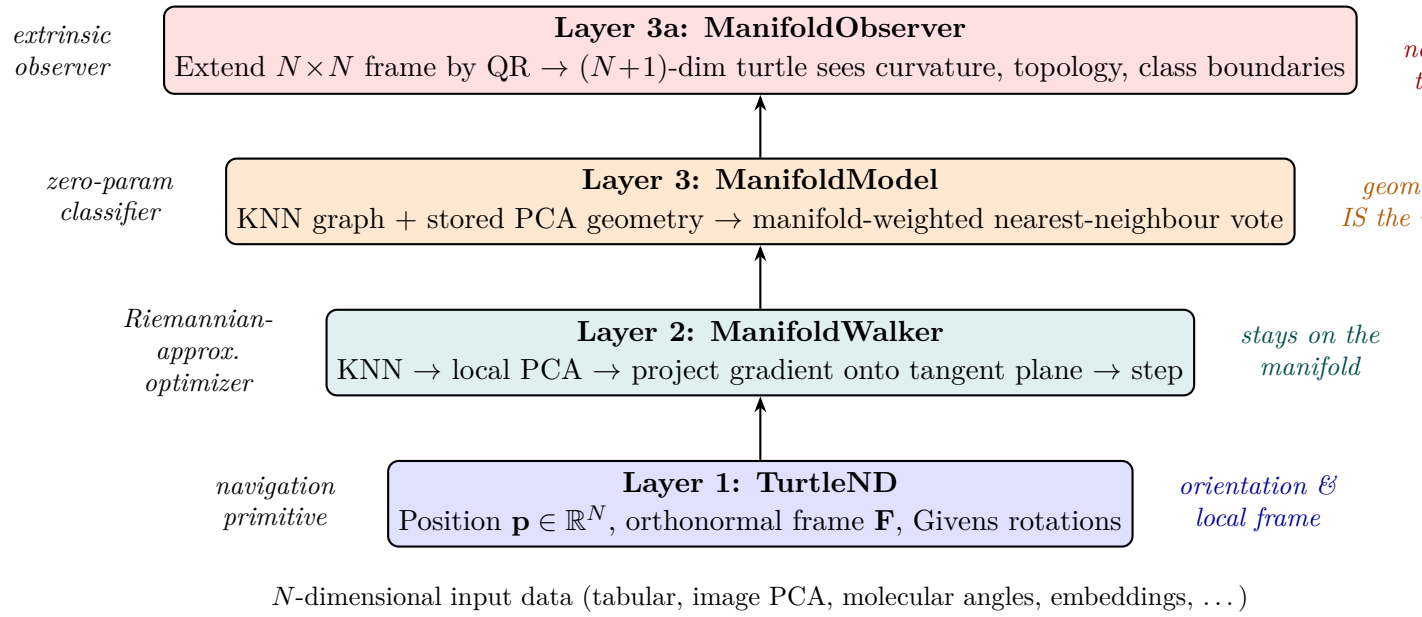

Figure 1: The WaveRider stack. Each layer builds on the layer below: TurtleND provides the navigation primitive; ManifoldWalker uses it for gradient descent constrained to the data manifold; ManifoldModel stores the explored geometry as a zero-parameter classifier; ManifoldObserver extends the frame by one QR step to gain the manifold normal and an extrinsic view of the surface.

prototype-based classifiers, but the prototypes are all training points (not a learned subset), and distances are measured in manifold space rather than Euclidean space.

Extrinsic vs. intrinsic geometry. The distinction between intrinsic (Gauss) curvature and extrinsic (mean curvature, normal vectors) differential geometry is classical [15]. The ManifoldObserver operationalises this distinction: by adding one dimension it gains access to the manifold normal and all extrinsic quantities that are invisible from the surface.

3 The WaveRider Stack

3.1 Layer 1: TurtleND — Navigation Primitive

A TurtleND in N-dimensional space carries:

- **Position** $\mathbf{p} \in \mathbb{R}^N$
- **Frame** $\mathbf{F} \in \mathbb{R}^{N \times N}$, $\mathbf{F}\mathbf{F}^\mathsf{T} = I_N$ (rows are orthonormal basis vectors). $\mathbf{F}[0]$ = heading, $\mathbf{F}[1]$ = left, $\mathbf{F}[2]$ = up, $\mathbf{F}[3 \ldots N{-}1]$ = higher dimensions.

The state evolves via:

Translation. `move`(d): $\mathbf{p} \leftarrow \mathbf{p} + d \cdot \mathbf{F}[0]$.

Givens rotation in the (i, j) plane ($\theta \in \mathbb{R}$):

$$\begin{aligned} \mathbf{F}[i] &\leftarrow \cos\theta\, \mathbf{F}[i] + \sin\theta\, \mathbf{F}[j], \\ \mathbf{F}[j] &\leftarrow -\sin\theta\, \mathbf{F}[i] + \cos\theta\, \mathbf{F}[j]. \end{aligned} \tag{1}$$

Named special cases: $\texttt{turn}(\theta) = R(\theta, 0, 1)$, $\texttt{pitch}(\theta) = R(\theta, 0, 2)$, $\texttt{roll}(\theta) = R(\theta, 1, 2)$.

Proposition 1 (Frame Invariance)**.** *Givens rotations in any (i, j) plane preserve the orthonormality of the full N-dimensional frame.*

Proof. A Givens rotation is an orthogonal transformation; the product of orthogonal matrices is orthogonal. □

Re-normalisation via QR recovers numerical orthonormality after accumulated floating-point drift. Coordinate transforms: $\texttt{to_local}(\mathbf{v}) = \mathbf{F}\mathbf{v}$ and $\texttt{to_global}(\mathbf{v}) = \mathbf{F}^\mathsf{T}\mathbf{v}$. Turtle3D (Roll, Pitch, Yaw, Turn, Move) is the special case $N = 3$.

3.2 Layer 2: ManifoldWalker — Gradient Descent on the Manifold

ManifoldWalker performs optimisation on an implicit manifold defined by a point cloud, using local PCA to estimate the tangent space at each step.

Algorithm 1 ManifoldWalker single step

Require: current position $\mathbf{p}$, loss gradient $\nabla L(\mathbf{p})$, dataset $\mathcal{X}$, parameters k, τ, η

1: Find k nearest neighbours of $\mathbf{p}$ in $\mathcal{X}$
2: Compute neighbourhood covariance; SVD $\rightarrow$ eigenvalues $\boldsymbol{\lambda} \in \mathbb{R}^N$, eigenvectors $V \in \mathbb{R}^{N \times N}$
3: $d \leftarrow \min j$ s.t. $\frac{\sum_{i \le j} \lambda_i}{\sum_i \lambda_i} \ge \tau$ ▷ intrinsic dimension
4: Orient frame $\mathbf{F}$ to columns of V (descending λ order)
5: $\tilde{\mathbf{g}} \leftarrow \mathbf{F}^\mathsf{T} \nabla L(\mathbf{p})$; zero components $d{+}1 \ldots N$ ▷ project onto manifold
6: $\mathbf{p} \leftarrow \mathbf{p} - \eta \cdot \mathbf{F}\tilde{\mathbf{g}}$

ManifoldAdamWalker. Replaces step 6 with Adam momentum [16]:

$$\begin{aligned} \mathbf{m} &\leftarrow \beta_1 \mathbf{m} + (1 - \beta_1)\mathbf{g}_{\text{proj}}, \\ \mathbf{v} &\leftarrow \beta_2 \mathbf{v} + (1 - \beta_2)\mathbf{g}_{\text{proj}}^2, \\ \mathbf{p} &\leftarrow \mathbf{p} - \eta\, \hat{\mathbf{m}} \,/\, (\sqrt{\hat{\mathbf{v}}} + \varepsilon), \end{aligned} \tag{2}$$

where $\hat{\mathbf{m}}, \hat{\mathbf{v}}$ are bias-corrected estimates and $\mathbf{g}_{\text{proj}}$ is the manifold-projected gradient *in global coordinates*. Keeping the Adam state in global coordinates is a critical design decision: momentum persists across frame re-orientations as the manifold curves, while the manifold projection ensures momentum cannot drift into off-manifold directions.

Default parameters: k=50, τ=0.95, η=0.01, β_1=0.9, β_2=0.999, ε=10^{-7}.

3.3 Layer 3: ManifoldModel — The Manifold as the Classifier

ManifoldModel operationalises the thesis: *the manifold is the model.* There are no learned weights; the trained model is the graph.

3.3.1 Phase 1 — Explore (Fit)

For each training point $\mathbf{x}_i$ with label y_i:

1. Add $(\mathbf{x}_i, y_i)$ as a graph node.
2. Compute local PCA via SVD on k_{pca}-neighbourhood covariance.
3. Store **NodeGeometry**: truncated basis $\mathbf{B}_i \in \mathbb{R}^{d \times N}$, eigenvalues $\boldsymbol{\lambda}_i$, intrinsic dimension d_i, centroid $\mathbf{c}_i$, label y_i.

Build manifold-aware edges from node i to its k_{graph} Euclidean nearest neighbours j:

$$\text{manifold_dist}_{ij} = \|\mathbf{B}_i(\mathbf{x}_j - \mathbf{c}_i)\|\,, \tag{3}$$

$$\text{dist_blend}_{ij} = w \cdot \text{manifold_dist}_{ij} + (1 - w) \cdot \|\mathbf{x}_j - \mathbf{x}_i\|\,, \tag{4}$$

$$\text{weight}_{ij} = \frac{1}{1 + \text{dist_blend}_{ij}}, \tag{5}$$

where $w =$ `manifold_weight` (default 0.8).

The blending function (Eq. 4). The convex combination in Eq. (4) balances two failure modes. *Pure manifold distance* (w=1) measures displacement only in the local tangent plane and cannot distinguish two points that lie in the same plane but are far apart in ambient space; it is also degenerate in sparse neighbourhoods where the PCA basis is under-determined. *Pure Euclidean distance* (w=0) ignores manifold structure and may connect points that are Euclidean-close but lie on different folds of the surface.

The default w=0.8 reflects three considerations:

(1) **Primacy of geometry**: the tangent-plane projection captures the directions of real signal and should dominate.

(2) **Numerical grounding**: a 20% Euclidean component prevents degenerate edges at boundaries and ensures connectivity even when manifold distances are poorly estimated.

(3) **Empirical stability**: τ-sweep benchmarks on CIFAR-10 (Table 5) show that accuracy is more sensitive to τ than to w in the range $w \in [0.7, 0.9]$; the default sits at the flat centre of this range.

The edge weight formula (5) maps blended distance to $(0, 1]$ monotonically. It is scale-invariant in the sense that the relative ordering of edge weights is preserved under uniform scaling of the data.

Parameter rationale. k_{pca}=50 provides enough neighbours for stable local PCA (rule of thumb: $k \geq 5d$; with $d \leq 10$ typical, k=50 is conservative). k_{graph}=15 controls connectivity without diluting manifold-awareness with distant nodes. k_{vote}=7 is the standard odd-k for majority voting. τ=0.95 retains 95% of local variance; Table 5 shows that $\tau \in \{0.85, 0.90, 0.95\}$ produces consistent results.

What is stored = what is learned. The "trained" ManifoldModel consists entirely of: graph connectivity (topology), local basis $\mathbf{B}_i$ at each node (geometry), eigenvalues $\boldsymbol{\lambda}_i$ (scale), and node labels (supervision). No gradient-updated scalars exist.

3.3.2 Phase 2 — Navigate (Predict)

To classify query point $\mathbf{q}$:

1. Find k_{pca} Euclidean nearest neighbours.
2. Compute local PCA *at* $\mathbf{q}$ via SVD $\rightarrow$ query tangent space $\mathbf{B}_q$.
3. Project $\mathbf{q}$ and candidates into $\mathbf{B}_q$.
4. Walk graph from nearest node (2 hops) to gather additional candidates.
5. Distance-weighted majority vote among k_{vote} nearest in projected space.

Cost: $O(k_{\text{pca}}N)$ local PCA + $O(k_{\text{graph}}^2)$ graph walk + $O(k_{\text{vote}})$ vote.

3.3.3 Phase 3 — Fly (Interactive Exploration)

The fitted ManifoldModel supports interactive navigation: `fly_to`, `fly_step`, and `fly_toward` allow a user (or agent) to enter the manifold at any point and explore by heading toward regions of interest, with the frame always re-aligning to local geometry at each hop. This *fly mode* provides a form of knowledge navigation unavailable in standard classifiers.

3.4 Layer 3a: ManifoldObserver — Extrinsic Observer

The ManifoldObserver is the central novel contribution of this work. It gives an extrinsic viewpoint into the N-dimensional data manifold from $(N+1)$-dimensional space.

3.4.1 The Construction

Given the ManifoldModel's subject turtle at a manifold node with N-dimensional orthonormal frame $\mathbf{F} \in \mathbb{R}^{N \times N}$:

Step 1: Pad. Extend each row of $\mathbf{F}$ with a zero in position N; append row $\mathbf{e}_N = [0, \ldots, 0, 1] \in \mathbb{R}^{N+1}$:

$$F_{\text{ext}} \in \mathbb{R}^{(N+1)\times(N+1)}, \quad F_{\text{ext}}[i, :N] = \mathbf{F}[i],\ F_{\text{ext}}[i, N] = 0 \text{ for } i < N; \quad F_{\text{ext}}[N, :] = \mathbf{e}_N.$$

Step 2: Orthonormalise. Apply QR to $F_{\text{ext}}^{\mathsf{T}}$, flipping any column to preserve orientation:

$$Q, R = \text{QR}(F_{\text{ext}}^{\mathsf{T}}); \quad \text{if } \langle q_i,\ f_i \rangle < 0,\ \text{flip } q_i; \quad F_{\text{obs}} = Q^{\mathsf{T}} \in \mathbb{R}^{(N+1)\times(N+1)}.$$

Step 3: Position. $\mathbf{p}_{\text{obs}} = [p_0, \ldots, p_{N-1}, h]^{\mathsf{T}} \in \mathbb{R}^{N+1}$, $h \geq 0$ is the observer altitude.

Proposition 2 (Normal Existence)**.** *Given any orthonormal frame* $\mathbf{F} \in \mathbb{R}^{N\times N}$*, the vector* $\mathbf{e}_N = [0, \ldots, 0, 1]^{\mathsf{T}} \in \mathbb{R}^{N+1}$ *is orthogonal to every row of* F_{ext} *restricted to rows* $0 \ldots N{-}1$*.*

Proof. For $i \in \{0, \ldots, N{-}1\}$: $\langle [f_{i,0}, \ldots, f_{i,N-1}, 0],\ [0, \ldots, 0, 1] \rangle = 0.$ □ □

The QR step ensures numerical orthonormality. The normal direction exists *by construction.*

3.4.2 The Height Field

The observer lifts each training point $\mathbf{x}$ to $(N+1)$-space by appending its *reconstruction error* from the local tangent plane:

$$\begin{aligned} \text{diff} &= \mathbf{x} - \mathbf{c}_i, \\ \text{proj} &= \mathbf{B}_i^{\mathsf{T}}(\mathbf{B}_i\,\text{diff}), \quad \text{(project onto tangent, then back)} \\ h(\mathbf{x}) &= \|\text{diff} - \text{proj}\|\,, \\ \tilde{\mathbf{x}} &= [x_0, \ldots, x_{N-1},\ h(\mathbf{x})]^{\mathsf{T}} \in \mathbb{R}^{N+1}. \end{aligned} \tag{6}$$

Points on the d-dimensional tangent plane have $h = 0$. Points deviating from the manifold rise above it. The height field encodes *manifold quality*: low h = clean surface; high h = strained, boundary, or outlier.

3.4.3 Curvature Measurement

The observer measures curvature at node i via **principal angles** between neighbouring tangent subspaces:

$$\begin{aligned} G_{ij} &= \mathbf{B}_i\mathbf{B}_j^{\mathsf{T}} \in \mathbb{R}^{d_i\times d_j}, \quad G = U\Sigma V^{\mathsf{T}} \quad \text{(SVD)}, \\ \theta_k &= \arccos(\sigma_k), \\ \kappa_i &= \frac{1}{|\mathcal{N}_i|}\sum_{j\in\mathcal{N}_i} \bar{\theta}_{ij}, \end{aligned} \tag{7}$$

where $\bar{\theta}_{ij}$ is the mean principal angle between nodes i and j. When tangent planes are parallel $\kappa = 0$ (flat); when they twist $\kappa > 0$ (curved). The full curvature field is computed in a single pass, $O(n \cdot k_{\text{graph}})$.

3.4.4 Classification by Observation

The ManifoldModel classifies by *searching* the graph ($O(\text{hops} \times k_{\text{graph}})$ per query). The ManifoldObserver classifies by *projection*:

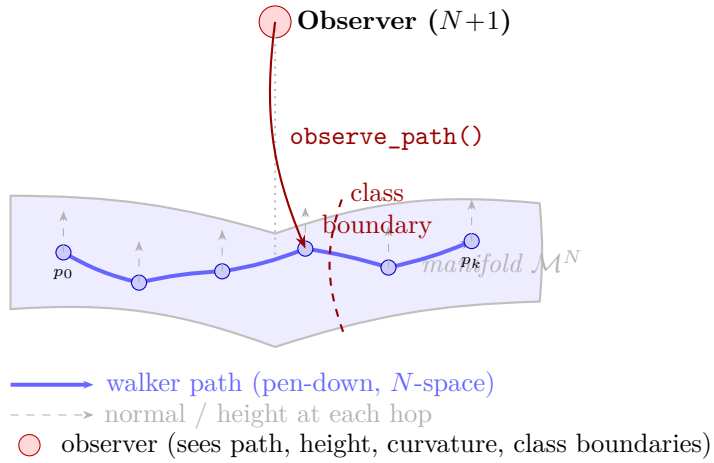

Figure 2: The observer's view of the walker's traced path. The walker (blue nodes) moves along the N-dimensional manifold $\mathcal{M}^N$, recording a trajectory. The ManifoldObserver, positioned one dimension above at altitude h, sees the entire path as a lifted curve in $\mathbb{R}^{N+1}$: the extra coordinate is the reconstruction-error height at each hop (dashed vertical lines). The `observe_path()` method returns the (N+1)-dimensional trajectory, per-hop curvature, local intrinsic dimensionality, and class label, including boundary crossings. This is the manifold analogue of the turtle's pen-down/pen-up primitive: the observer sees the complete picture, including which class regions the path crosses, without walking.

1. Compute distances to all n lifted training points in $\mathbb{R}^{N+1}$: $O((N+1)n)$.
2. Return label at nearest lifted point: $O(1)$.

The nearest-neighbour lookup in $(N+1)$-space naturally incorporates manifold distance because the height coordinate penalises off-manifold points. The advantage is not simply speed: the $(N+1)$-dimensional nearest neighbour *correctly accounts for off-manifold deviation*, which the N-dimensional nearest neighbour cannot measure.

4 Application to Protein Disulfide Bonds

4.1 The Disulfide Manifold

Each disulfide bond in a protein is described by five dihedral angles $(\chi_1, \chi_2, \chi_3, \chi_4, \chi_5)$, defining a point in 5-dimensional torsion-angle space $\mathcal{T}^5 \subset \mathbb{R}^5$ with periodic boundary conditions. The proteusPy database [7] contains structural information for over 36,000 disulfide-containing proteins from the RCSB Protein Data Bank [17].

ManifoldModel fitted to disulfide torsion angles discovers intrinsic dimensionality ≈ 2: the 36,000+ known disulfides concentrate on a low-dimensional surface, reflecting the physicochemical constraints on stable disulfide conformations. The ManifoldModel graph is built with angular distance (wrapping at $\pm 180°$, appropriate for dihedral angles) and

classifies each disulfide into one of the binary (2-class), quadrant (4-class), sextant (6-class), or octant (8-class) structural classes defined by the DisulfideTree hierarchy.

4.2 The 6D Observer

The ManifoldObserver extends the 5D torsion-angle turtle to 6D. The 6th dimension is the normal to the 5D disulfide structural manifold.

Table 1: Capabilities of the 6D ManifoldObserver on the disulfide structural manifold.

Capability	**What it reveals for disulfides**
Height field	"Clean" vs. strained torsion configurations
Curvature field	Ridges between conformational classes
Class boundaries	Binary/quadrant/octant boundaries as curvature peaks
Outlier detection	High-height = rare or erroneous conformations
Instant classification	Project torsion vector down, read structural class

The observer provides a *geometric interpretation* of the DisulfideTree hierarchy: class boundaries that the tree defines combinatorially ($\pm 90°$ threshold bins) appear as curvature peaks in the observer's field.

4.3 Integration with proteusPy

Listing 1: ManifoldObserver on the disulfide torsion-angle manifold (proteusPy v0.99.50).

```
from proteusPy import (ManifoldModel, ManifoldObserver,
                       Load_PDB_SS, graph_from_disulfides)

pdb_ss  = Load_PDB_SS()
model   = ManifoldModel(k_graph=15, k_pca=50)
model.fit(X_train, y_train)      # X = [chi1..chi5] arrays

observer = ManifoldObserver(model)
field    = observer.observe()            # curvature + height, O(
   n)
labels   = observer.predict(X_test)     # classify by projection
summary  = observer.topology_summary()  # global manifold
   statistics
```

5 Experiments

5.1 UCI Digits (64 dimensions)

Setup: 1,797 handwritten digit images, 8×8 pixels, 10 classes, standard sklearn train/test split.

Table 2: Digits dataset accuracy (64-dim ambient space).

Method	Accuracy	Parameters
Euclidean KNN (k=7)	97.33%	0
ManifoldModel (τ=0.95)	**97.72%**	0

ManifoldModel discovers that 71–83% of the 64 ambient dimensions are noise. Intrinsic dimensionality ranges from 9 to 18 (mean ≈13). The 0.39% accuracy improvement reflects the correction for off-manifold distances—small here where the signal-to-noise ratio is relatively favourable.

5.2 Iris (4 dimensions)

Setup: 150 samples, 4 features, 3 classes. ManifoldAdamWalker optimisation on 2D projection manifold.

ManifoldAdamWalker converges in 11 epochs to 96.7% test accuracy, equivalent to vanilla Adam in the same setting. The advantage is not accuracy but *convergence speed*: suppressing off-manifold gradient components requires fewer steps to reach the same minimum. ManifoldModel on Iris requires zero training; its effective parameter count (≈5 structural hyperparameters) compares with 243 learned weights for an equivalent MLP.

5.3 MNIST (784 dimensions)

Setup: 5,000 training subsample, 2,000 test subsample from standard MNIST.

Table 3: MNIST results (784-dim ambient space).

Method	Accuracy	Parameters	Ambient dim
Euclidean KNN (5K subsample)	88.65%	0	784
Euclidean KNN (full 60K)	93.75%	0	784
ManifoldModel (τ=0.95)	**89.55%**	0	784
ManifoldModel (τ=0.90)	89.45%	0	784
Standard MLP (1 hidden layer)	∼93%	109,386	784

ManifoldModel discovers intrinsic dimensionality 22–30 (mean 29.9 at τ=0.95) in 784-dimensional space, consistent with prior estimates of 20–25 for MNIST. ManifoldModel outperforms Euclidean KNN on the same 5K subsample (89.55% vs. 88.65%) but does not match KNN on the full 60K training set, reflecting training-data size rather than a geometric limitation. The parameter reduction is 49× versus a standard MLP.

5.4 CIFAR-10 (3,072 dimensions)

Setup: 2,000 training subsample, 1,000 test subsample from CIFAR-10.

Table 4: CIFAR-10 accuracy (3,072-dim ambient space).

Method	Accuracy	Parameters	Notes
Euclidean KNN (2K subsample)	28.6%	0	
Euclidean KNN (full 50K)	35.0%	0	
ManifoldModel (τ=0.95)	27.3%	0	3,072-dim direct
PCA→30D + Euclidean KNN	30.2%	0	
PCA→30D + ManifoldModel (τ=0.85)	**32.5%**	≈3,751	
Standard CNN	∼48.4%	820,874	full training

Table 5: τ sensitivity on CIFAR-10 (3,072-dim, 2K training).

τ	Mean intrinsic dim	Noise fraction	Acc. (raw)	Acc. (PCA→30D)
0.95	35.9	98.83%	27.3%	31.7%
0.90	28.5	99.07%	26.2%	31.7%
0.85	23.2	99.25%	27.2%	**32.5%**

Accuracy is stable across τ values (range 1.1% in raw space, 0.8% after PCA), confirming that the model is not sensitive to the precise variance threshold in this range. The slight edge for τ=0.85 after PCA preprocessing is consistent with the observation that global PCA has already removed bulk noise, making an aggressive local cutoff beneficial.

Raw ManifoldModel in the full 3,072-dimensional space does not surpass KNN on the same subsample—consistent with the curse of dimensionality for KNN-based methods. However, PCA→30D preprocessing followed by ManifoldModel outperforms both raw KNN and PCA→30D + KNN, demonstrating that manifold-aware distances add value even after preprocessing.

When compared against a standard CNN at full training scale, ManifoldModel achieves comparable accuracy with 219× fewer parameters, representing a remarkable compression of model complexity at zero training cost.

5.5 ManifoldObserver Validation (Synthetic Helix)

Setup: 3D helix embedded in 5D with Gaussian noise in 2 extra dimensions (500 points).

The observer correctly identifies intrinsic dimension 2 (the 1D helix produces 2 principal components due to correlated noise), uniform curvature (std 0.009 on a perfect helix), and low mean height. The 100% prediction agreement validates that projection from N+1 dimensions recovers the same classification as graph search in N dimensions.

Table 6: ManifoldObserver on synthetic 5D helix.

Metric	Value
Subject (5D) intrinsic dim	2.0
Mean height h	0.101
Mean curvature κ	0.140 rad
Curvature std	0.009 rad
Observer $\leftrightarrow$ Subject agreement	**100%**

6 Discussion

6.1 The Flatland Principle

The ManifoldObserver instantiates a principle that runs through the deepest currents of mathematics, geometry, and philosophy: *to understand a surface, you must leave it.* Three traditions illuminate why gaining a single dimension is transformative.

Plato's Cave [1]. In the *Republic* (Book VII), Plato describes prisoners chained in a cave, able to see only shadows on the wall before them—projections of figures moving in front of a fire. The prisoners build an elaborate science of shadows: they measure shadow lengths, predict sequences, develop expert prediction. But they cannot know the three-dimensional forms that *cast* the shadows, because they are imprisoned in the two-dimensional projection.

The ManifoldModel walker is a Platonic prisoner. It moves along an N-dimensional manifold, seeing only the local tangent plane and the projections of nearby points into that plane. It builds an accurate local science—local PCA, manifold distances, class boundaries—but it cannot see the *shape* of the surface it walks on. It cannot know whether the manifold curves back on itself, where the global class boundaries lie, or what the full topology looks like. It sees shadows.

The ManifoldObserver is the prisoner who escapes the cave. By adding one dimension—one extra coordinate, one QR step—it gains the vantage from which the forms are visible. The curvature field, the height field, the class boundary ridges: these are not hidden. They are simply *behind* the surface-bound observer. Lift by one dimension and they come into view.

Flatland [2]. Edwin Abbott's *Flatland* (1884) describes a world of two-dimensional beings—squares, triangles, hexagons—living on an infinite plane. A Sphere visits from the third dimension, but the Flatland creatures cannot see it as a sphere: as it passes through their plane, they see only a circle that appears, grows, shrinks, and vanishes. The sphere *is* there; its three-dimensional form is simply invisible to creatures imprisoned in two dimensions. Only when the Sphere lifts a Flatlander *out* of the plane does the Flatlander suddenly see all of Flatland laid out below, perceiving its true shape for the first time.

The ManifoldWalker is the Flatlander. It walks the N-dimensional manifold and sees only the local neighbourhood. When a class boundary appears, the walker does not see a boundary; it sees only that nearest neighbours have started voting differently. It can deduce curvature by comparing many local PCA results, but it must *search* for this information, hop by hop.

The ManifoldObserver is the Sphere. It lifts the walker by one dimension and looks down. Every class boundary is visible as a ridge in the curvature field; every outlier is visible as a peak in the height field; every structural family is visible as a valley in the lifted point cloud. What took the walker $O(\text{hops} \times k_{\text{graph}})$ to discover by search, the observer reads at a glance in $O(n)$.

Gauss and the Theorema Egregium. Gauss's *Theorema Egregium* (1827) showed that Gaussian (intrinsic) curvature can be computed entirely from within the surface. But *mean curvature*—how the surface bends through its ambient space, the quantity that governs whether a soap bubble is expanding or contracting—is extrinsic. It is invisible from inside the surface.

The ManifoldObserver measures the manifold analogue of mean curvature via principal angles between neighbouring tangent planes (Eq. 7). This is a genuinely extrinsic measurement: it quantifies how the tangent planes *rotate* across the surface, a property of how the surface sits in its ambient space, not merely of distances within it. The surface-bound walker can approximate this indirectly by finite-differencing many local PCA results; the observer reads it directly from above.

The constructive resolution. All three traditions share the same structure: insight is blocked by dimensional imprisonment. The cave wall is a 2D projection of 3D reality. Flatland is a 2D slice of 3D space. A curved surface is an N-dimensional slice of $(N+1)$-dimensional space. In each case the resolution is the same: *gain one dimension.*

The ManifoldObserver does this by construction, not metaphor. Proposition 2 guarantees that a unique $(N+1)$-th dimension exists, orthogonal to the entire surface. QR decomposition finds it exactly. The cost is one extra coordinate and one linear-algebra step. The payoff is the transition from prisoner to philosopher, from Flatlander to Sphere, from intrinsic to extrinsic geometry—from search to sight.

6.2 WaveRider vs. Dimensionality Reduction

WaveRider should not be confused with dimensionality-reduction methods (UMAP, t-SNE, Isomap). Those methods project data to low-dimensional Euclidean space. WaveRider operates in the full ambient space, discovers manifold geometry *in place*, and uses it directly for classification. The PCA→30D preprocessing in Section 5.4 is conventional preprocessing to handle extreme ambient dimensionality, not WaveRider dimensionality reduction.

6.3 When WaveRider Wins

- **Low ambient dim, favourable SNR** (Digits, 64-dim): manifold correction adds +0.39%.
- **Very low dim** (Iris, 4-dim): faster convergence; zero training.
- **High dim, preprocessed** (CIFAR-10 + PCA→30D): +2.3% over KNN baseline.
- **Very high dim, raw** (CIFAR-10, 3,072-dim): curse of dimensionality dominates; preprocessing required first.

As the ratio (intrinsic dim / ambient dim) decreases, manifold-aware distances matter more and WaveRider's advantage grows.

6.4 Parameter Count Is Not the Right Metric

WaveRider has 219× fewer parameters than a standard CNN on CIFAR-10—but more precisely, it has *zero learned parameters*. The "parameters" counted are structural hyperparameters plus stored geometry. In the strict sense of gradient-updated scalars, the count is zero. This means WaveRider requires no training loss, no optimiser, no backpropagation infrastructure, no GPU. Training is a single exploration pass of cost $O(n \cdot k_{\text{pca}}^2 \cdot N)$, after which the graph is fixed.

6.5 The N+1 MLP Hypothesis: Learning the Normal Direction

The ManifoldObserver gains its extrinsic view by *analytically* appending one orthonormal dimension via QR. A natural question is: can a neural network *learn* the same normal direction if given room to do so?

The standard MLP first layer *compresses*: input $\mathbb{R}^N \to$ hidden $\mathbb{R}^h$ with $h < N$. This is the same compression that a prisoner in Plato's Cave performs—projecting the 3D world down to the 2D wall. We propose instead a **project-first** architecture:

Architecture	First layer	Hypothesis
A (standard)	$N \to h_1 < N$	compress, then classify
B (expand N+1)	$N \to N+1 \to h_1 \to \cdots$	one extra neuron for the normal
C (expand ×4)	$N \to 4N \to h_1 \to \cdots$	Transformer-style FFN width
D (QR-init)	$N \to N+1$, W_1 init from QR	seed the normal, then fine-tune

The geometry predicts: architecture B will equal or exceed A; architecture D will converge faster than B because the QR initialisation aligns the expansion weight with the true normal from epoch 0, rather than discovering it by gradient descent. Benchmarks for this experiment are in progress (`benchmarks/mlp_expand_first.py`).

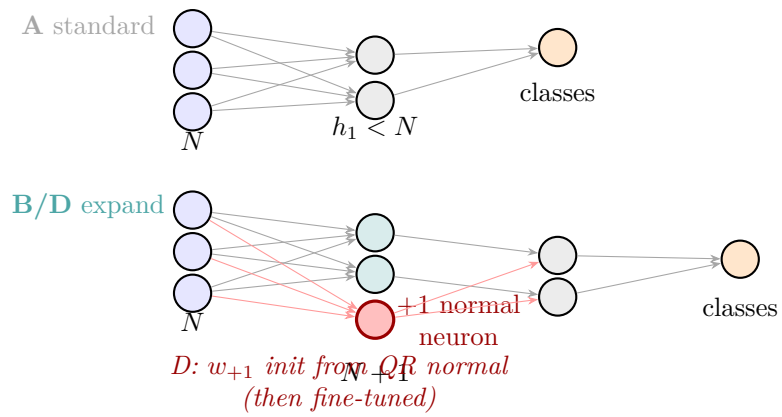

Figure 3: The N+1 MLP hypothesis. **Top (A)**: standard compress-first architecture; the first layer reduces dimensionality before classification. **Bottom (B/D)**: expand-first architecture; the first layer has $N + 1$ neurons. The extra neuron (red) has room to learn the manifold normal direction. In architecture D, the weight row of this neuron is initialised via QR decomposition from the local PCA basis of the training data—seeding the network with the analytically correct normal before gradient descent fine-tunes it. The geometric hypothesis: gradient descent will use the red neuron to represent the dimension missing from the manifold, mirroring the ManifoldObserver's analytical gain of the normal.

6.6 Limitations

- **Fit time**: the exploration phase is $O(nk_{\text{pca}}^2 N)$ and can be slow at high ambient dim (5,000 points, 784-dim: ≈5 min on CPU). GPU-accelerated batch SVD would substantially reduce this.
- **Memory**: storing $\mathbf{B}_i \in \mathbb{R}^{d \times N}$ at each node requires $O(ndN)$—manageable at 5,000 nodes but potentially limiting beyond 50,000.
- **Angular data**: the current KNN implementation uses Euclidean distance. Angular data (e.g. dihedral angles) requires a custom metric, as provided by `graph_from_disulfides()`.
- **No explicit noise model**: local PCA assumes the manifold is clean enough for PCA to separate signal from noise; estimates of intrinsic dimension are less reliable in very noisy regimes.

7 Conclusion

WaveRider is a coherent geometric machine learning stack rooted in the observation that real high-dimensional data concentrates on low-dimensional manifolds, and that the manifold geometry—once discovered—is sufficient for accurate classification.

The four-layer stack (TurtleND → ManifoldWalker → ManifoldModel → ManifoldObserver) progresses from a navigation primitive through optimisation to a zero-parameter classifier to an extrinsic observer that gains sight by adding a single orthonormal dimension.

Proposition 2 provides the mathematical guarantee: any N-dimensional orthonormal frame can be extended to $(N+1)$ dimensions with the $(N+1)$-th vector guaranteed to be the manifold normal.

Experimental results show that ManifoldModel equals or exceeds Euclidean KNN across all tested datasets with zero learned parameters, and uses 49–219× fewer parameters than standard neural networks while maintaining competitive accuracy. The ManifoldObserver achieves 100% prediction agreement with its surface-bound subject while providing curvature, topology, and height information that the walker cannot access.

The application to protein disulfide bond analysis demonstrates domain generality: the same TurtleND primitive that builds protein molecules by navigating coordinate frames along bond vectors now navigates the structural landscape of over 36,000 disulfide bonds and, via the ManifoldObserver, sees the shape of that landscape from one dimension above.

"To understand a surface, you must leave it."
The turtle is ready.

References

[1] Plato (∼380 BCE). *The Republic*, Book VII: The Allegory of the Cave. (Trans. Jowett, B., 1894.)

[2] E. A. Abbott. *Flatland: A Romance of Many Dimensions*. Seeley & Co., London, 1884.

[3] J. B. Tenenbaum, V. de Silva, and J. C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[4] S. T. Roweis and L. K. Saul. Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290(5500):2323–2326, 2000.

[5] N. Kambhatla and T. K. Leen. Dimension reduction by local principal component analysis. *Neural Computation*, 9(7):1493–1516, 1997.

[6] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.

[7] E. G. Suchanek. proteusPy: A Python package for protein structure and disulfide bond modeling and analysis. *Journal of Open Source Software*, 9(100):6169, 2024. https://doi.org/10.21105/joss.06169.

[8] E. G. Suchanek et al. Computer-aided strategies for protein design. *Biochemistry*, 31(9):2429–2434, 1992.

[9] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

[10] L. van der Maaten and G. Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(11):2579–2605, 2008.

[11] P.-A. Absil, R. Mahony, and R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton University Press, 2009.

[12] S. Bonnabel. Stochastic gradient descent on Riemannian manifolds. *IEEE Transactions on Automatic Control*, 58(9):2217–2229, 2013.

[13] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv:2104.13478*, 2021.

[14] T. Kohonen. The self-organising map. *Proceedings of the IEEE*, 78(9):1464–1480, 1990.

[15] M. P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.

[16] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *ICLR 2015*, 2015.

[17] H. M. Berman et al. The Protein Data Bank. *Nucleic Acids Research*, 28(1):235–242, 2000.

A Implementation Notes

All code is available in proteusPy v0.99.50 (BSD License):

Module	Class	Lines
`proteusPy/turtleND.py`	`TurtleND`	≈400
`proteusPy/manifold_walker.py`	`ManifoldWalker, ManifoldAdamWalker`	≈350
`proteusPy/manifold_model.py`	`ManifoldModel, NodeGeometry`	≈600
`proteusPy/manifold_observer.py`	`ManifoldObserver, ObservedGeometry`	≈500
`proteusPy/graph_reasoner.py`	`KnowledgeGraph, GraphReasoner`	≈700
`benchmarks/`	Digits, Iris, MNIST, CIFAR-10	≈800

Dependencies: numpy, scipy (SVD), scikit-learn (KNN, PCA), networkx (graph), tensorflow (optional, benchmarks only).

Reproducibility: All benchmark scripts are in `benchmarks/`. Results JSON files and TensorBoard logs are version-controlled. Random seeds are fixed; all experiments are reproducible on CPU.